

Práctica N.º 3

Estado compartido con Provider, MultiProvider y Selector

Asignatura	Desarrollo de Aplicaciones Móviles (INF 662)	•	6.º semestre
Docente	M. Sc. Lic. Huáscar Fedor Gonzales Guzmán		
Base	Flutter (canal estable) • Dart 3 • paquete provider ^6.x (único externo)		
Modalidad	Individual (trabajo independiente)	•	Fecha de entrega: 05 / 10 / 2026

En esta práctica aplicarás lo trabajado en las Guías de Laboratorio 9 y 10 de la asignatura: la gestión del estado compartido con el paquete provider. El estado que en la práctica anterior vivía dentro de un StatefulWidget pasará a modelos que extienden ChangeNotifier, se expondrá a toda la aplicación y se consumirá desde varias pantallas, optimizando las reconstrucciones con Selector. Desarrollarás de forma individual las tres aplicaciones que se describen a continuación.

01 Objetivos

- Modelar el estado compartido en clases que extienden `ChangeNotifier` y notifican sus cambios con `notifyListeners()`.
- Exponer el estado con `ChangeNotifierProvider` y `MultiProvider` en la raíz de la aplicación.
- Consumir el estado con `context.watch`, `context.read` y `Consumer`, distinguiendo **cuándo reconstruir** y cuándo solo invocar acciones.
- Optimizar las reconstrucciones de la interfaz con `Selector` y su parámetro `child`.
- Mantener **varias pantallas sincronizadas** sin pasar el estado por constructor ni devolverlo con `Navigator.pop`.

02 Contenidos que se evalúan

La práctica se limita a los siguientes contenidos. No utilices sqflite, shared_preferences ni bases de datos: la persistencia se abordará en la práctica siguiente. Los datos viven en memoria.

ALCANCE

Provider: paquete provider ^6.x (flutter pub add provider), ChangeNotifier, notifyListeners(), ChangeNotifierProvider, MultiProvider.

Consumo del estado: context.watch<T>(), context.read<T>(), Consumer<T>, Selector<T, S> con el parámetro child.

Base de prácticas anteriores: Scaffold, ListView.builder, GridView.builder, Card, Navigator, Form / TextFormField con validación, SnackBar, ThemeData y themeMode.

03 Instrucciones generales

- Trabajo individual. Cada estudiante entrega su propia solución; el código copiado invalida la calificación de las partes involucradas.
- Un proyecto Flutter por aplicación (o un repositorio con una carpeta por app, nombradas `app1_mibiblioteca`, `app2_cinebutaca`, `app3_habitudiario`).
- El **estado compartido** debe vivir en modelos `ChangeNotifier` expuestos con `Provider`; `setState` solo se permite para estado local efímero (p. ej. un formulario).
- Separa los modelos de las pantallas: cada modelo en su propio archivo dentro de `lib/models/` y las pantallas en `lib/screens/`.
- El único paquete externo permitido es `provider`; agrégalo con `flutter pub add provider`.
- Formatea el código con `dart format` y usa nombres de variables, clases y widgets descriptivos.
- Incluye por cada app una captura de pantalla y un breve README (qué hace y cómo ejecutarla).

04 Aplicaciones a desarrollar

Desarrolla las tres aplicaciones. Cada una vale 30 puntos (ver criterios de evaluación). Los retos opcionales otorgan puntaje adicional (bonus) hasta un máximo de 10 puntos sobre el total.

APP 1

MiBiblioteca — Catálogo con lista de lectura compartida

Objetivo. Compartir un mismo estado entre varias pantallas con `ChangeNotifierProvider`, sin pasar datos por constructor.

REQUISITOS MÍNIMOS

- Un **modelo de datos** Libro (título, autor, género y año) y una clase `BibliotecaModel` que extienda `ChangeNotifier` con la lista de lectura y los métodos agregar, quitar y contiene.
- Registrar `BibliotecaModel` con `ChangeNotifierProvider` en la raíz, **por encima de** `MaterialApp`.
- Pantalla de catálogo con `ListView.builder` (al menos 10 libros); cada elemento tiene un botón para agregar o quitar de la lista que invoca el modelo con `context.read`.
- En el `AppBar`, un contador con la **cantidad de libros en la lista**, actualizado con `context.watch`, que navega a la pantalla «Mi lista».
- Pantalla «Mi lista» construida con `Consumer<BibliotecaModel>`: muestra los libros, permite quitarlos y maneja el **estado vacío**.
- Al volver al catálogo, los botones reflejan los cambios hechos en «Mi lista» **sin** usar `Navigator.pop` con resultado.

Widgets clave

`CONTEXT.READ`

`CHANGENOTIFIER`

`CONSUMER`

`NOTIFYLISTENERS`

`CHANGENOTIFIERPROVIDER`

`CONTEXT.WATCH`

ADVERTENCIA

No uses `context.watch` dentro de un `onPressed`: en los callbacks se usa `context.read`. Expón la lista del modelo como solo lectura (`List.unmodifiable`) para que nadie la modifique sin notificar.

RETO OPCIONAL (BONUS)

Permite marcar libros como **leídos** desde «Mi lista» y muestra el avance (leídos / total) con un `LinearProgressIndicator`.

APP 2

CineButaca — Reserva de asientos con Selector

Objetivo. Optimizar las reconstrucciones de la interfaz con `Selector`, reconstruyendo solo la parte que cambia.

REQUISITOS MÍNIMOS

- Un `SalaModel` que extienda `ChangeNotifier` con una matriz de asientos (al menos 6 filas × 8 columnas), cada uno **libre**, **seleccionado** u **ocupado** (usa un enum `EstadoAsiento`), un precio por asiento y un **máximo de 6 asientos** por reserva.
- Sala dibujada con `GridView.builder`; cada asiento se envuelve en un `Selector<SalaModel, EstadoAsiento>` para que al tocarlo **solo se reconstruya ese asiento**.
- Barra inferior con la cantidad de asientos seleccionados y el total a pagar, usando `Selector` y el parámetro `child` para la parte estática (texto fijo o botón).
- Si el usuario intenta superar el máximo, mostrar un `SnackBar` y no seleccionar el asiento.
- Pantalla de confirmación que lee el resumen desde el modelo; al confirmar, los asientos seleccionados pasan a **ocupados**, la selección se limpia y se vuelve a la sala.

Widgets clave

`SELECTOR`

`PARÁMETRO CHILD`

`ENUM`

`GRIDVIEW.BUILDER`

`CONTEXT.READ`

`SNACKBAR`

ADVERTENCIA

Tocar un asiento ocupado no debe cambiar su estado ni el total. El botón de confirmar permanece deshabilitado mientras no haya asientos seleccionados.

RETO OPCIONAL (BONUS)

Agrega categorías de asiento (normal y VIP) con precios distintos y una leyenda de colores. Incluye un `debugPrint` en el builder de cada asiento para demostrar que solo se reconstruye el que cambia.

APP 3

HábitoDiario — Seguimiento de hábitos con MultiProvider

Objetivo. Coordinar dos estados independientes con `MultiProvider` y reflejar los cambios de configuración en toda la aplicación.

REQUISITOS MÍNIMOS

- Un `HabitosModel` (`ChangeNotifier`) con la lista de hábitos (nombre, categoría y cumplido hoy), métodos para agregar y alternar el cumplimiento, y un getter con el **porcentaje del día**.
- Un `PreferenciasModel` (`ChangeNotifier`) con el nombre del usuario, el modo de tema claro u oscuro y una **meta diaria** (número de hábitos).

- Registrar ambos modelos con `MultiProvider`; el `themeMode` de `MaterialApp` se controla desde `PreferenciasModel`.
- Formulario validado (`Form` + `TextFormField` + `DropdownButtonFormField`) para agregar hábitos: nombre no vacío y categoría obligatoria; al guardar, escribe en el modelo con `context.read` y vuelve a la lista.
- Pantalla principal con saludo por nombre, lista de hábitos con `CheckboxListTile`, porcentaje del día y un aviso visible cuando se **alcanza la meta**.
- Pantalla de ajustes (nombre, `Switch` de tema y meta validada con `int.tryParse`, mayor que cero) cuyos cambios se reflejan **al instante** en toda la app.

Widgets clave

`MULTIPROVIDER`

`CHANGENOTIFIER`

`THEMEMODE`

`CONSUMER`

`CONTEXT.WATCH`

`FORM`

ADVERTENCIA

El tema y el saludo deben actualizarse sin reiniciar la app. Si un cambio solo aparece al volver a abrir una pantalla, el modelo no está llamando a `notifyListeners()`.

RETO OPCIONAL (BONUS)

Elimina hábitos deslizando con `Dismissible` y muestra un **resumen por categoría** (cumplidos / total) calculado con un getter del modelo.

05 Producto entregable

- Código fuente de las tres aplicaciones (proyectos Flutter o carpetas dentro de un repositorio).
- Una captura de pantalla por aplicación mostrando su funcionamiento (pantallas sincronizadas, sala con selección, confirmación, ajustes, etc.).
- Un README por aplicación con una descripción breve y las instrucciones de ejecución.

06 Criterios de evaluación

Cada aplicación se evalúa sobre 30 puntos con la siguiente distribución. La calificación final es el promedio de las tres aplicaciones.

Crterio	Descripción	Puntos
Funcionalidad	Cumple todos los requisitos mínimos del enunciado.	10
Uso de Provider	Modelos <code>ChangeNotifier</code> bien definidos, uso correcto de <code>watch</code> / <code>read</code> / <code>Consumer</code> / <code>Selector</code> y estado sincronizado entre pantallas.	7
Interfaz	Diseño claro y ordenado: layout, espaciado y legibilidad.	3
Casos límite	Estado vacío, entradas inválidas y límites (máximo de asientos, meta diaria).	5
Código	Organización y legibilidad: modelos separados de pantallas, nombres, dart format.	5
Total por aplicación		30

NOTA

Los retos opcionales se acreditan como bonus (hasta 10 puntos) solo si los requisitos mínimos de la aplicación están completos. Una app que resuelve el estado compartido con `setState` o pasando datos por constructor, en lugar de `Provider`, no obtiene el puntaje de «Uso de `Provider`».

07 Forma de entrega

- Sube tu solución a un repositorio de GitHub personal (público o con acceso al docente) y envía el enlace en un archivo de texto a la plataforma virtual de la universidad.
- Verifica que cada proyecto compile con `flutter run` y que su `pubspec.yaml` incluya la dependencia `provider` antes de entregar.